



UnDF - UNIVERSIDADE DO DISTRITO FEDERAL
CIÊNCIA DA COMPUTAÇÃO

BRUNO DIOGO
CARLOS VICTOR
KAUÃ MACIEL
MARIA PAULA FERREIRA
PIETRA CARDOSO
RAÍSSA CARVALHO

UNIPROJETOS DASHBOARD

Projeto prático de desenvolvimento de software apresentado como requisito parcial
de avaliação da disciplina de Projeto Aplicado IV - Design
Professora: Carolina Gonçalves

BRASÍLIA-DF 2026

RELATÓRIO TÉCNICO: UNIPROJETOS DASHBOARD

I. USUÁRIA E PROBLEMA: CONHECENDO A MARIANA

A Mariana é a coordenadora do Laboratório de Inovação Digital da UnDF e a pessoa responsável por fazer a ponte entre os professores que precisam de bolsistas e os alunos que buscam projetos. A rotina dela é super corrida e cheia de interrupções, com professores na porta cobrando o preenchimento de vagas e alunos mandando e-mails com dúvidas.

Devido a essa pressão constante para resolver as coisas rápido, a Mariana se apegou muito às planilhas manuais pela alta previsibilidade que elas oferecem. Ela sabe exatamente em qual coluna cada dado está, o que lhe dá uma forte sensação de controle e segurança psicológica. Ao pensar em adotar um sistema novo, a maior prioridade dela é a estabilidade e a clareza. Ela precisa de uma ferramenta que mostre as informações de forma imediata, dispensando um tempo extra de aprendizado no seu dia a dia já apertado.

O processo atual, o cenário de falha e a tensão de uso. O fluxo de trabalho dela começa quando um professor pede a um aluno de Ciência da Computação com urgência. Hoje, ela abre uma planilha pesada, aplica filtros coluna por coluna e conta as linhas e as vagas na mão. Após cruzar os dados com dificuldade, ela precisa sair do documento e abrir o seu e-mail padrão para tentar entrar em contato com o estudante ideal. Esse processo manual consome muito tempo. A lentidão operacional resulta em candidatos bons aguardando respostas por dias e vagas preenchidas fora do prazo.

Apesar de a ferramenta atual ser muito ineficiente e causar a perda desses prazos, Mariana hesita em abandoná-la justamente porque a planilha lhe dá total previsibilidade. Essa tensão entre a ineficiência do processo e o conforto do que é familiar orienta todo o projeto: o novo painel precisa ser mais rápido para resolver o gargalo de tempo, mas deve ser transparente em todos os seus estados para que ela sinta a mesma segurança psicológica ao trocar de ferramenta.

Como essa história define a Arquitetura da Informação do nosso painel:

Para fazer a transição da planilha para o nosso painel ser bem-sucedida, a nova interface precisa conquistar a confiança da Mariana nos primeiros segundos de uso. O sistema deve poupar o esforço mental dela, substituindo a ineficiência por velocidade, mantendo a transparência sobre o que está acontecendo na tela. Essas características orientaram as seguintes decisões de prioridade:

- Hierarquia de bate-pronto: Ela precisa entender o cenário do laboratório em menos de 10 segundos. Por isso, a primeira coisa que o painel exibe no topo é um resumo claro com o total de projetos abertos e o total de inscrições cadastradas.

- Filtros como protagonistas: Os filtros de área temática e status ganham grande destaque na tela inicial. O objetivo é permitir o cruzamento de dados com apenas dois cliques, garantindo a rapidez que a planilha não tem.
- Informação sob demanda (sem poluição): Os dados detalhados dos inscritos (nome, curso, turno e data) ficam guardados em um segundo nível de navegação. Essas informações mais densas aparecem exclusivamente quando ela seleciona um projeto específico, mantendo a tela inicial limpa e focada.
- Foco na previsibilidade (Estados da Interface): Para substituir o conforto que ela sentia na planilha, a interface mantém a coordenadora informada sobre tudo o que acontece, criando um ambiente seguro. Durante o carregamento dos dados, o sistema exibe um skeleton screen para sinalizar que o conteúdo está a caminho. Caso ela filtre uma área sem projetos correspondentes, o painel avisa claramente "Nenhum projeto encontrado" e oferece um botão rápido para limpar os filtros. Em situações de falha na busca dos dados, o sistema apresenta uma mensagem amigável explicando o problema e um botão sugerindo tentar novamente.

II. ARQUITETURA DA INFORMAÇÃO E DECISÕES DE DESIGN

O painel foi construído utilizando tecnologias nativas web (Vanilla Stack), garantindo máxima performance sem a dependência de frameworks externos. A estruturação baseou-se em HTML5 Semântico focado em acessibilidade e CSS3 Avançado (utilizando variáveis globais, Grid Layout e Flexbox) para garantir um layout 100% responsivo em múltiplos dispositivos. A hierarquia visual do painel foi desenhada em formato Top-Down (do macro para o micro), focado em reduzir a carga cognitiva da coordenadora Mariana e poupar seu esforço mental frente à ineficiência das planilhas manuais.

As seguintes decisões de prioridade orientaram o design:

- Hierarquia de Bate-Pronto: A coordenadora precisa entender o cenário do laboratório em menos de 10 segundos. Por isso, no topo, isolamos os KPIs (Total de Vagas, Inscrições e Ocupação). No nível intermediário, os filtros e a pesquisa ganham protagonismo visual, permitindo o cruzamento de dados com poucos cliques. No nível inferior, a tabela exibe as informações essenciais dos projetos.
- Filtro Estratégico (Semestre): Adicionamos proativamente o filtro de Semestre à interface. Essa decisão de design visa atender à necessidade primária da coordenadora descrita no cenário: isolar e visualizar rapidamente apenas os projetos pertinentes ao período letivo atual, poupando tempo de triagem.

- **Dados sob demanda (Barra Lateral):** Para evitar a poluição visual, os dados detalhados dos inscritos (nome, curso, turno e data) ficam guardados em um segundo nível de navegação. A decisão de substituir um modal central padrão por uma Barra Lateral (Drawer) deslizante foi tomada para manter o contexto: a coordenadora consegue ler os dados do aluno sem perder a visão da tabela de projetos no fundo ofuscado.
- **Ordenação Dinâmica de Dados:** Para agilizar a localização de informações específicas, implementamos a ordenação interativa (crescente e decrescente) nas colunas da tabela principal e na lista de alunos dentro da Barra Lateral. Essa flexibilidade permite que a usuária reorganize a visualização de acordo com a prioridade do momento (como ordenar os projetos com mais vagas ou agrupar os candidatos por curso), facilitando a leitura e a tomada de decisão.
- **Feedback de Estados e Previsibilidade:** Para garantir a segurança psicológica da usuária ao sair das planilhas manuais, mapeamos todos os estados do sistema. Aplicamos microinterações adaptativas (efeitos de hover e transições suaves) e um Modo Escuro (Dark Mode) que preserva rigorosamente as taxas de contraste e critérios de acessibilidade (WCAG).
- **Tratamento de Exceções Visuais:** Implementamos esqueletos de carregamento (Loading) e tratativas visuais amigáveis para dois cenários distintos de exceção. Para buscas sem correspondência (Filtro Sem Resultado), a interface explica o ocorrido e oferece o botão de recuperação Limpar Filtros. Já para erros críticos de rede ou falha na leitura do banco de dados (Falha na Conexão), o sistema exibe uma mensagem técnica clara acompanhada do botão Tentar Novamente, garantindo que a coordenadora não fique travada na tela.

III. ESTRUTURA DOS DADOS (MODELAGEM DO MOCK.JSON)

A modelagem dos dados foi projetada visando o uso de Vanilla JS e o consumo via Fetch API nativo. O banco de dados simulado (mock.json) utiliza uma arquitetura orientada a documentos.

- **Nó Global (dashboard):** Um objeto destinado a armazenar as métricas consolidadas e KPIs (como vagas totais e ocupadas). Isso simula o comportamento de uma API real que já entrega os dados prontos para o topo da tela.
- **Nó Iterável (projetos):** Um array de objetos onde cada projeto atua como um documento com propriedades primitivas de identificação.
- **Aninhamento de Inscrições:** A decisão de modelagem mais importante foi aninhar o sub-array de alunos diretamente dentro do objeto do seu respectivo projeto. Essa abordagem hierárquica otimiza o frontend: o JavaScript consome a árvore relacional em uma única requisição e acessa os dados dos alunos inscritos instantaneamente ao abrir a Barra Lateral, sem a necessidade de lógicas complexas de Join.

IV. TESTE DE USABILIDADE

1. Teste Pessoa 1

O teste de usabilidade foi realizado com um usuário externo. Inicialmente, o projeto foi apresentado de forma ampla, sem o fornecimento de detalhes operacionais da interface ou instruções durante a execução, com o objetivo de observar a análise geral e verificar se o participante compreenderia a finalidade do painel livremente.

1.1 Execução das Tarefas e Registro de Atritos

Tarefa	Tempo de Execução	Conclusão (Sim/Não)	Principal Ponto de Atrito
Participante 1: Tarefa 1 (Encontrar vagas de Projetos de Engenharia)	01m 45s	Sim	Utilizou imediatamente a barra de pesquisa livre por texto e tentou diversas vezes sem sucesso. Demorou a perceber a existência do filtro lateral de "Área Temática".
Participante 1: Tarefa 2 (Encontrar dados de contato do aluno)	00m 45s	Sim	Concluiu sem grandes dificuldades. Constatou que projetos com nomes extensos ficavam cortados na lista, atrito que foi superado ao clicar na linha para abrir os detalhes.

1.2 Melhorias Propostas (A partir da observação)

- Filtros: Adicionar uma legenda visual mais clara para indicar a finalidade do uso dos filtros laterais.
- Interatividade: Inserir um indicador visual ou legenda sinalizando que a linha inteira do projeto é clicável para exibir a lista de estudantes.

1.3 Feedback Visual e Conclusão Geral

- Acessibilidade Visual (Tema Escuro): O participante pontuou que o contraste das cores no tema escuro é um pouco forte, especificamente no card que contém o gráfico em forma de pizza. Para a usuária final, que precisará utilizar o sistema durante todo o dia, esse nível de contraste pode se tornar visualmente cansativo a longo prazo.
- Avaliação Final: Após a explicação completa sobre a persona e o problema resolvido pela interface, o usuário considerou o sistema bom e intuitivo. Ele ressaltou que, com um breve período de adaptação e uso, conseguiria operar o painel sem problemas.

2. Teste Pessoa 2

O teste de usabilidade foi conduzido com um segundo participante, também externo ao desenvolvimento do projeto. Assim como no teste anterior, a interface foi apresentada sem explicações prévias sobre sua estrutura ou funcionalidades, permitindo que o usuário explorasse o sistema livremente. O foco desta sessão foi observar a curva de aprendizado e a eficiência na localização de informações específicas relacionadas à área de Ciência da Computação.

2.1 Execução das Tarefas e Registro de Atritos

Tarefa	Tempo de Conclusão	Conclusão (Sim/Não)	Principal Ponto de Atrito
Participante 2: Tarefa 1 (Encontrar vagas de Ciência da Computação)	02m 10s	Sim	Levou algum tempo até notar o filtro lateral e selecionar a opção "Ciência da Computação" na lista de áreas temáticas. Relatou que os nomes das áreas poderiam estar mais destacados visualmente.
Participante 2: Tarefa 2 (Encontrar dados de contato do aluno)	01m 15s	Sim	Conseguiu localizar as informações, mas mencionou que a disposição dos dados de contato (e-mail e telefone) não estava imediatamente visível na visualização resumida do projeto. Foi necessário abrir os detalhes completos, o que gerou um pequeno atrito por conta da expectativa de encontrar esses dados de forma mais direta.

2.2 Melhorias Propostas (A partir da observação)

- Acesso a Dados de Contato: Inserir um atalho ou ícone visual (ex.: envelope e telefone) diretamente na linha do projeto na lista, permitindo que o participante visualize as informações de contato sem precisar acessar a visualização detalhada.
- Organização da Lista: Ajustar a largura da coluna de títulos dos projetos para evitar cortes desnecessários, já que muitos projetos de Ciência da Computação possuem nomes técnicos extensos

2.3 Feedback Visual e Conclusão Geral

- Navegação Geral: O usuário avaliou positivamente a lógica geral do sistema, mencionando que, mesmo com os pequenos atritos iniciais, a interface é coerente e as informações são bem organizadas após o primeiro contato.

- **Avaliação Final:** O participante concluiu que o sistema atende bem ao propósito de conectar alunos e projetos na área de Ciência da Computação. Ele ressaltou que as melhorias sugeridas podem tornar a experiência mais fluida, especialmente para novos usuários, mas que, no geral, o painel é funcional e de aprendizado rápido. Recomendou ainda que uma breve dica interativa (tooltip) pudesse ser exibida na primeira utilização para guiar o usuário sobre a existência dos filtros laterais.

V. REVISÃO GERAL

O projeto “Uniprojetos Dashboard” atingiu com sucesso o objetivo de modernizar e otimizar o fluxo de trabalho da coordenação do Laboratório de Inovação Digital. A transição de planilhas manuais para uma Single Page Application (SPA) responsiva e acessível demonstrou ganhos significativos em performance e usabilidade.

Pontos Fortes Destacados:

- **Performance:** A escolha por Vanilla JavaScript e a ausência de frameworks pesados resultaram em um carregamento quase instantâneo e interações fluidas.
- **Acessibilidade:** A implementação rigorosa de HTML semântico e atributos WAI-ARIA garante que o sistema seja inclusivo e navegável por teclado e leitores de tela.
- **Design Centrado no Usuário:** A arquitetura da informação focada em “Hierarquia de BatePronto” e “Dados sob demanda” reduz a carga cognitiva da usuária principal (Mariana).

Oportunidades de Melhoria Futura:

- **Integração com Backend Real:** Substituir o mock.json por uma API RESTful ou GraphQL conectada a um banco de dados relacional para gerenciar dados em tempo real.
- **Exportação de Relatórios:** Adicionar a funcionalidade de exportar a lista de alunos inscritos em formato CSV ou PDF diretamente da Barra Lateral.
- **Sistema de Notificações:** Implementar alertas visuais ou por e-mail quando um novo aluno se inscrever em um projeto prioritário

VI. REPOSITÓRIOS E ACESSO AO SISTEMA

- Repositório (Código-fonte e Pull Requests):

<https://github.com/DiogoBrito777/dashboard-undf>

- Deploy de Produção (GitHub Pages):

<https://diogobrito777.github.io/dashboard-undf>

VII. ANEXO - DOCUMENTAÇÃO TÉCNICA

1. Arquitetura e Stack

O projeto é uma Single Page Application (SPA) construída com tecnologias nativas do lado do cliente (Front-end), operando no navegador através da manipulação do DOM.

- Marcação: HTML5 Semântico provido de atributos dinâmicos WAI-ARIA para suporte a Leitores de Tela.
- Estilização: CSS3, orientado a variáveis globais (:root) para suporte a temas de cores e gerenciamento estrutural por Flexbox e Grid.
- Lógica: Vanilla JavaScript (ES6+) aplicado à manipulação do DOM e interceptação de eventos.
- Consumo de Dados: API Fetch assíncrona com promessas (async/await).
- Dependência Externa: Chart.js (via CDN) para renderização visual e dinâmica em canvas.

2. Estrutura de Arquivos

A separação de responsabilidades aplica o padrão estrutural MVC simplificado para front-end:

- index.html: Camada de Visão (View). Define a estrutura do DOM, a semântica e os atributos de acessibilidade (como aria-live="polite" e role="dialog"), e provê os contêineres para injeção visual dos dados.
- style.css: Camada de Apresentação. Gerencia regras de layout macro, tipografia, paletas de cores avaliadas para contraste seguro e estabilidade adaptativa por quebra de linhas (flex-wrap).
- script.js: Camada de Controle (Controller/Logic). Atua no gerenciamento de variáveis de estado em memória, ouve eventos interativos de UI (mouse e teclado) e orquestra as atualizações das tabelas e gráficos.
- mock.json: Camada de Dados (Model). Banco de dados estático baseado em documentos estruturados com as informações cadastrais de 28 projetos iteráveis, nós de agregações e arrays aninhados.

3. Gerenciamento de Estado (JavaScript)

O estado da aplicação reside em variáveis globais dentro do escopo principal do script.

Essas variáveis operam como a única fonte da verdade (Single Source of Truth) para as rotinas de renderização:

- `projetosOriginais`: Array estático em memória que preserva o retorno do banco de dados após o carregamento assíncrono inicial.
- `projetosFiltrados`: Array mutável gerado pelos resultados lógicos de filtros cruzados e reordenações dinâmicas. Entrega as informações vitais para as funções de desenho da Tabela e dos Gráficos.
- `sortConfig`: Objeto estruturado para rastrear a seleção métrica atual da ordenação e a direção de ordenação (variável `asc`: booleano).
- `currentOpenProjectId`: Inteiro guardado na memória operacional para mapear com exatidão o projeto exibido na Barra Lateral de detalhes.
- `lastFocusedElement`: Ponteiro de armazenamento dedicado ao elemento que detinha o foco de navegação antes da abertura do modal, garantindo sua devolução ordenada e acessível ao fechá-lo.

4. Fluxo Lógico e Funções Principais

4.1. Boot e Fetch de Dados (`carregarDados`)

Função assíncrona encarregada da partida estrutural da aplicação. Ela aciona o componente visual de carregamento (Spinner), executa o `fetch('mock.json')` e engatilha uma latência (`API_DELAY_MS`) para garantir o ciclo de vida visual e de transição. Após a promessa ser resolvida, ela popula as variáveis globais de estado e dispara a renderização. Erros de resposta de rede resultam em processamento no bloco `catch`, que desenha a interface vermelha de erro (Error State) na tela principal.

4.2. Motor de Busca Multidimensional (`processarFiltros`)

A função analítica de filtro cruza simultaneamente os inputs de texto digitados (nome e área) com seleções estabelecidas nos comboboxes (Área Temática, Status do Projeto e Semestre).

- **Normalização Estrita**: Aplica os métodos `toLowerCase()`, `normalize('NFD')` e substituição Regex (`/[\u0300-\u036f]/g`) nas variáveis de string para propiciar buscas isentas das particularidades de capitalização e acentuação gráfica originais.
- **Debounce Pattern**: A sub-rotina `aplicarFiltrosComLoading` intercepta as respostas a eventos do usuário e envia o agendamento da renderização em um `setTimeout(FILTER_DELAY_MS)`. O timer é intencionalmente reiniciado (`clearTimeout`) a cada detecção de alteração nos inputs ou teclado, ativando as modificações visuais na conclusão

do estímulo de digitação.

4.3. Regras de Negócio e Cálculos (atualizarCards)

A atualização analítica dos Indicadores-Chave (KPIs) utiliza métodos funcionais avançados de varredura de array (reduce). Projetos cujo conteúdo da variável status detenha a string "encerrado" insere o número 0 na contagem métrica de vagas disponíveis. Este critério estabelecido evita o reflexo cumulativo das edições finalizadas de projetos sob a capacidade de lotação atual do dashboard.

4.4. Acessibilidade Computacional (WAI-ARIA)

O sistema integra padrões de usabilidade motora e leitura de tela assistiva em sua arquitetura de eventos.

- Navegação Teclada: Todas as entidades de projetos dispostas na Tabela recebem a atribuição `tabindex="0"` e a escuta nativa dos comandos das teclas "Enter" e "Espaço", efetuando a abertura exata da respectiva Barra Lateral sem demandar uso do ponteiro. Botões de select e o campo de texto disparam filtros via detecção do botão "Enter".
- Gerenciamento do Foco: O processo de sobreposição de tela move o direcionamento de input do usuário ao comando de exclusão (Botão de Fechar) no surgimento da Barra Lateral. O esvaziamento deste modal, via clique do elemento específico ou do acionamento direto da tecla "Escape" (Esc), devolve o cursor funcional à linha tabular onde a experiência se iniciou.

5. Padrões de Interface e Estilo (CSS)

5.1. Theming (Light/Dark Mode)

A progressão cromática de temas opera via CSS acionada diretamente pelo JavaScript. O comando `script.js` alterna a nomenclatura funcional do atributo `data-theme="dark"` retida no elemento raiz. A cascata de estilos contém um fragmento de design com seletor genérico `[data-theme="dark"]` responsável pela sobrescrita dos identificadores Custom Properties presentes em `:root`, gerando a repintura das tonalidades do documento estrutural em tela.

5.2. Layout Adaptativo e Estabilidade

- Responsividade Macro: A tecnologia CSS Grid atua nos grandes contêineres arquitetônicos (tais como `.stats-grid` e `.main-content-grid`), fornecendo comportamento lógico bidimensional frente a modificações nas dimensões do viewport.
- Responsividade Micro e Controles: Flexbox ordena e centraliza o escopo interno de

múltiplos micro-componentes. O setor direcional de filtros incorpora alinhamentos flexíveis compostos pelo elemento normativo flex-wrap: wrap, providenciando reordenações limpas das seleções para novas linhas e preservando a totalidade de extensão dos rótulos expostos.

6. Modelo de Dados (JSON Schema)

A representação física basilar do sistema provém do arquivo local (mock.json), alinhado a uma arquitetura baseada hierarquicamente em objetos e documentos estruturados sequencialmente.

- **Nó Global (dashboard):** Atua como objeto formatado para acondicionar o somatório dos numerais avaliados da infraestrutura integral.
- **Nó Iterável (projetos):** Matriz funcional populada pelos 25 arquivos de origem individuais dos respectivos grupos acadêmicos de trabalho. Estabelece as categorias em modelo "chave e valor" com a taxonomia informacional (nomenclatura, semestre e escopo temático).
- **Nó Aninhado (alunos):** Estrutura alocada internamente à totalidade matricial das entidades listadas em "projetos". Consiste em bibliotecas indexadas correspondentes aos detalhes individuais vinculados aos inscritos operantes. O motor front-end efetua toda a transcrição inicial destes campos de dados e converte estas listagens operacionais diretamente para manipulação ramificada (tabela consolidada panorâmica e tela pormenorizada de inscrições).

7. Fluxogramas e Jornada do Usuário

A arquitetura da aplicação define caminhos de processamento e interação estruturados em duas perspectivas: a lógica interna de processamento e as etapas de navegação interativa.

7.1. Fluxograma de Sistema (System Flow)

O processamento interno segue uma cadeia linear de eventos lógicos:

- **Inicialização:** O navegador carrega o DOM e dispara o ouvinte principal (DOMContentLoaded).
- **Requisição Assíncrona (Fetch):** A função carregarDados() solicita a leitura do arquivo base de dados.
- **Resolução e Tratamento:** Com sucesso na leitura, os dados populam o array projetosOriginais. Falhas de rede direcionam a execução para o bloco de captura (catch), desenhando a interface de erro estrutural.
- **Renderização Base:** O sistema constrói as linhas da tabela em HTML, gera os gráficos no

elemento canvas via Chart.js e preenche os cálculos matemáticos dos KPIs superiores.

- Escuta Contínua (Listen): O motor lógico entra em estado de espera, monitorando eventos nos botões de filtro, barra de pesquisa ou navegação por teclado.
- Ciclo de Atualização: Modificações nos campos de busca ativam o ciclo `processarFiltros()`. O sistema processa o debounce, filtra a matriz e aciona a repintura das informações visuais com base no array `projetosFiltrados`.

7.2. Caminho do Usuário (User Journey)

O fluxo de interface direciona a navegação para análise macro e detalhamento sob demanda (Drill-down).

- Análise de Painel (Overview): O usuário inicia a sessão e recebe os indicadores de capacidade de atendimento, taxa percentual de ocupação e a distribuição gráfica das áreas acadêmicas.
- Filtragem e Segmentação: A operação da barra de pesquisa textual e dos seletores associados (Área Temática, Status e Semestre) segmenta a matriz de dados para localizar ocorrências exatas.
- Aprofundamento de Dados (Drill-down): O usuário interage com uma linha específica na tabela central utilizando o mouse ou os acionadores de teclado (teclas Enter ou Espaço).
- Inspeção de Inscrições: A Barra Lateral (Drawer) realiza o movimento de sobreposição. A interface entrega os KPIs isolados do projeto escolhido e a listagem relacional contendo nome, e-mail, telefone, curso e turno de cada aluno.
- Encerramento e Continuidade: O usuário executa o comando de fechamento do modal (clique no botão superior ou tecla Esc). O sistema encerra a sobreposição e retorna o foco de atenção para o elemento de origem, permitindo o avanço linear da navegação.

8. Declaração de Uso de Inteligência Artificial

Conforme as diretrizes do projeto, declaramos o uso da IA Gemini (Google) como ferramenta de apoio à programação durante o desenvolvimento. A atuação da ferramenta foi limitada a consultas de sintaxe da Fetch API, auxílio na formulação das Expressões Regulares (Regex) do motor de busca e esclarecimento de regras do W3C para os atributos de acessibilidade WAI-ARIA. As definições de arquitetura da informação, a modelagem dos dados e as decisões de design de interface foram concebidas e desenvolvidas pela própria equipe, com base nos requisitos do projeto.